

Reviewer Pack — Hook-Length Dim of Tseitin T-Join Partition Lower-Bounds Tre...

Entry a677f9fc0b8d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 18:40:33 UTC

Hook-Length Dim of Tseitin T-Join Partition Lower-Bounds Tree-Res

Entry ID: a677f9fc0b8d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 18:40:33 UTC

1. Conjecture

Field A (mathematical branch): Symmetric-group Specht-module dimensions via Frame–Robinson–Thrall hook-length formula (Frame–Robinson–Thrall 1954; James–Kerber 1981; Sagan 2001 ‘The Symmetric Group’). The hook-length functional $\dim S^\lambda = n! / \prod_{c \in \lambda} h(c)$ on integer partitions is a representation-theoretic dimension count from algebraic combinatorics and a canonical GCT-flavored invariant in the Mulmuley–Sohoni / Bürgisser–Ikenmeyer plethysm tradition. Applied here SYNTACTICALLY to the canonical T-join path-length partition $\pi(G, \omega)$ of a charged 3-regular graph, the invariant uses only integer factorials and divisions on a small partition (no ring/ F_q polynomial extension — factorials and hook quotients are not preserved under polynomial ring lifts), so the bridge is Aaronson–Wigderson algebrization-safe. The invariant is STRUCTURAL on (G, ω) (two Tseitin formulas with identical Boolean function $\equiv \text{FALSE}$ can have very different π), shielding from Razborov–Rudich natural-proofs. An arXiv search for ‘hook length formula’ AND (‘Tseitin’ OR ‘resolution refutation’ OR ‘proof complexity’) returns 0 direct hits and <5 adjacent papers (all Specht/plethysm work in GCT or on permutation patterns, never on charged 3-regular Tseitin partitions). Distinct from blacklisted Specht-isotype mass on perm-vs-det c_σ vectors (an isotype projection $\chi^\lambda \cdot c(f)$ on the permutation-monomial coefficient vector of homogeneous polynomials, NOT a hook product on a graph-theoretic partition), distinct from Eulerian-descent entropy / Tamari–Pallo rank / Patience-sort LIS / RSK shape concentration, and distinct from all prior Tseitin-side spectral, modulus, resistance, holonomy, Szegedy, fatgraph, and curvature attempts. **Field B** (complexity object): Tree-like Resolution / DPLL refutation size $t(T(G, \omega))$ for Tseitin XOR formulas on connected 3-regular graphs G with odd charge $\omega : V(G) \rightarrow F_2$ (Urquhart 1987; Ben-Sasson–Wigderson 2001; Alekhnovich–Razborov 2001 expansion regime), in the Cook–Reckhow–Krajíček bounded-arithmetic framework where $\neg T(G, \omega)$ is Σ^b_0 and $\log_2 t$ lower-bounds V^0 -witnessing complexity — the canonical Frege-depth / Extended-Frege stepping stone targeted by the PROOF_COMPLEXITY_TSEITIN approach.

Statement:

For every connected 3-regular graph G on n vertices with odd charge $\omega : V(G) \rightarrow F_2$, define the T-join path partition $\pi(G, \omega)$ as the multiset of edge-lengths of the canonical greedy minimum-weight T-join decomposition (T = odd-charge vertices, parity-fixed by appending the lex-smallest non-charged vertex when $|T|$ is odd; pairing is greedy BFS-closest pairs by lex tie-break), and let $\rho(G, \omega) := \log_2(\dim S^{\{\pi(G, \omega)\}})$ with $\dim S^{\{\pi(G, \omega)\}}$ computed by the Frame–Robinson–Thrall hook-length formula on π . Then there exists a universal

constant $c \geq 0.15$ such that for every (G, ω) with $|E(G)| \leq 40$ the tree-Resolution refutation size $t(T(G, \omega))$ satisfies $\log_2 t(T(G, \omega)) \geq c \cdot \rho(G, \omega) / \log_2(n+2)$. One counterexample (G, ω) with the inequality violated falsifies the conjecture.

Rationale (proposer’s reasoning):

Tseitin tree-Resolution lower bounds in the Ben-Sasson-Wigderson framework are controlled by how the odd charge spreads through the graph: dispersed charges force long path obligations that resolution must witness. The T-join path partition $\pi(G, \omega)$ captures exactly this spread, and the hook-length dimension $\dim S^\pi$ — a representation-theoretic, GCT-flavored coarsening — concentrates on partitions with many roughly equal parts (which is precisely the regime where Tseitin is provably hard) and collapses for clustered charges (where Tseitin is easy via local cancellation). This bridges the algebraic-combinatorial machinery used in the perm-vs-det / GCT_DET_PERM program (Specht modules, hook formulas, plethysm) with the proof-complexity hardness of expander-Tseitin, an angle no prior literature appears to have explored.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6b528a89c3e6ba1b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 120 instances ($n \in \{6, 8, 10, 12\}$, 30 seeds each), compute $r(G, \omega) = (\log_2 t^*(T(G, \omega))) \cdot \log_2(n+2) / \rho(G, \omega)$ for every instance with $\rho > 0$. Conjecture is SUPPORTED iff $\min r \geq 0.15$ over all 120 instances; FALSIFIED iff any single instance yields $r < 0.15$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.88	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - hook length formula Tseitin resolution refutation - Specht module dimension proof complexity lower bound - T-join partition 3-regular tree resolution Tseitin

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.7s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict, deque

def generate_3_regular_graph(n, seed):
    random.seed(seed)
    if n % 2 != 0:
        return None
    edges = []
    stubs = list(range(n)) * 3
    while stubs:
        u = random.choice(stubs)
        stubs.remove(u)
        v = random.choice([x for x in stubs if x != u])
        stubs.remove(v)
        edges.append((u, v))
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    visited = set()
    stack = [0]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            stack.extend(adj[node])
    if len(visited) != n:
        return None
    return adj

def compute_t_join_partition(adj, omega):
    T = [v for v in adj if omega[v] == 1]
    if len(T) % 2 != 0:
        T.append(min(v for v in adj if v not in T))
    pairs = []
    while T:
        u = T.pop()
        min_dist = float('inf')
        v = None
        for candidate in T:
            dist = bfs_distance(adj, u, candidate)
            if dist < min_dist or (dist == min_dist and candidate < v):
                min_dist = dist
                v = candidate
        if v is not None:
            T.remove(v)
            pairs.append((u, v))
    path_lengths = []
    for u, v in pairs:
        path_lengths.append(bfs_distance(adj, u, v))
    return path_lengths
```

```

def bfs_distance(adj, start, end):
    if start == end:
        return 0
    visited = {start: 0}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbor in adj[node]:
            if neighbor not in visited:
                visited[neighbor] = visited[node] + 1
                if neighbor == end:
                    return visited[neighbor]
            queue.append(neighbor)
    return float('inf')

def hook_length_dimension(partition):
    if not partition:
        return 0
    n = sum(partition)
    m = len(partition)
    if m == 0:
        return 0
    log_dim = 0
    for i, part in enumerate(partition):
        for j in range(1, part + 1):
            hook = part - j + (m - i)
            log_dim += math.log2(j) - math.log2(hook)
    return log_dim

def compute_resolution_size(adj, omega, max_nodes=2**18):
    n = len(adj)
    clauses = []
    for v in adj:
        neighbors = adj[v]
        clause = []
        for u in neighbors:
            clause.append((u, v))
        clauses.append(clause)
    for v in adj:
        if omega[v] == 1:
            clauses.append([(v, v)])
    num_vars = n
    num_clauses = len(clauses)
    if num_clauses == 0:
        return 1
    nodes_explored = 0
    queue = deque()
    queue.append([[], []])
    while queue and nodes_explored < max_nodes:
        assignment, unit_clauses = queue.popleft()
        nodes_explored += 1
        satisfied = [False] * num_clauses
        for i, clause in enumerate(clauses):
            for lit in clause:
                if lit in assignment or (-lit[0], -lit[1]) in assignment:
                    satisfied[i] = True

```

```

        break
    if all(satisfied):
        return nodes_explored
    for i, clause in enumerate(clauses):
        if not satisfied[i]:
            unsat_lits = [lit for lit in clause if lit not in assignment and (-lit[0], -lit[1])]
            if len(unsat_lits) == 1:
                new_assignment = assignment + [unsat_lits[0]]
                new_unit_clauses = unit_clauses + [i]
                queue.append((new_assignment, new_unit_clauses))
    return max_nodes

def run_trial(seed):
    n_values = [6, 8, 10, 12]
    results = []
    for n in n_values:
        adj = None
        while adj is None:
            adj = generate_3_regular_graph(n, seed)
            seed += 1
        omega = [random.randint(0, 1) for _ in range(n)]
        if sum(omega) % 2 != 1:
            omega[0] = 1 - omega[0]
        partition = compute_t_join_partition(adj, omega)
        rho = hook_length_dimension(partition)
        if rho == 0:
            continue
        t_star = compute_resolution_size(adj, omega)
        if t_star == 0:
            continue
        log_t_star = math.log2(t_star)
        log_n = math.log2(n + 2)
        r = (log_t_star * log_n) / rho
        results.append({
            "n": n,
            "rho": rho,
            "log_t_star": log_t_star,
            "log_n": log_n,
            "r": r,
            "conjecture_holds": r >= 0.15,
            "counterexample": "" if r >= 0.15 else f"n={n}, rho={rho}, log_t_star={log_t_star}, log_n={log_n}, r={r}"
        })
    if not results:
        return {
            "metric_name": "r",
            "metric_value": 0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No valid instances generated"
        }
    min_r = min(r["r"] for r in results)
    worst_case = min(results, key=lambda x: x["r"])
    return {
        "metric_name": "r",
        "metric_value": min_r,
        "instances_tested": len(results),

```

```

        "conjecture_holds": min_r >= 0.15,
        "counterexample": worst_case["counterexample"] if min_r < 0.15 else ""
    }

if __name__ == "__main__":
    seeds = sys.argv[1:] if len(sys.argv) > 1 else [random.randint(1, 1000000) for _ in range(30)]
    seeds = [int(seed) for seed in seeds]
    metric_values = []
    conjecture_holds = []
    counterexamples = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        metric_values.append(trial["metric_value"])
        conjecture_holds.append(trial["conjecture_holds"])
        if trial["counterexample"]:
            counterexamples.append(trial["counterexample"])
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(conjecture_holds) / len(conjecture_holds)
    if counterexamples:
        print(f"RESULT: FALSIFIED counterexample=\"{counterexamples[0]}\" first_failing_seed={seeds[0]}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'r', 'metric_value': -59.79470570797252, 'instances_tested': 3, 'conjecture_holds':
1.0, log_t_star=18.0, log_n=3.321928094887362, r=-59.79470570797252'}

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0eec2ac0.py", line 188, in <module>
    trial = run_trial(seed)
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0eec2ac0.py", line 139, in run_trial
    adj = generate_3_regular_graph(n, seed)
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0eec2ac0.py", line 25, in generate_3_regular_graph
    v = random.choice([x for x in stubs if x != u])
            ~~~~~

```

```

File "/usr/lib/python3.12/random.py", line 347, in choice
    raise IndexError('Cannot choose from an empty sequence')

```

IndexError: Cannot choose from an empty sequence

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` in `generate_3_regular_graph` before completing the pre-registered 120 instances; only 3 instances ran and the reported counterexample has $\rho=-1.0$, which violates the $\rho>0$ precondition of the falsification criterion. Neither support nor falsification can be unambiguously established from this run. | next: Fix the 3-regular graph generator (handle the stub-pairing dead-end by restarting or using a configuration-model with retries) and re-run the full 120-instance p

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	?	?	0	0	1209379
2	propose	claude_max	opus	0	0	127716
3	preregistration	claude_max	opus	0	0	5747
4	novelty	claude_max	opus	0	0	4953
5	novelty	claude_max	opus	0	0	7795
6	test_gen	mistral	codestral-latest	0	0	319559
7	test_gen	mistral	codestral-latest	0	0	316494
8	test_gen	mistral	codestral-latest	0	0	323063
9	test_gen	mistral	codestral-latest	0	0	321812
10	judge	claude_max	opus	0	0	5076

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 2641594 ms total latency. Provider mix: {'?': 1, 'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/a677f9fc0b8d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a677f9fc0b8d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a677f9fc0b8d.tar.gz` (if generated)